Which Computation Answers the Physics Question?

- Start with the required physical output
- Choose among methods already learned
- Finish with a defensible capstone result

1. Physical Question

What is the temperature $T(t)$ of the body over time?

Newton cooling model

$$\frac{dT}{dt} = -\frac{(T - T_{env})}{\tau}$$

$T_{env} = 20$ °C

$T_0 = 80$ °C at $t = 0$ s

$\tau = 100$ s

$0 \le t \le 200$ s

Required physical output:
$T(t = 200\text{ s})$ in °C

2. Computation

Use a method already learned and within its limitation

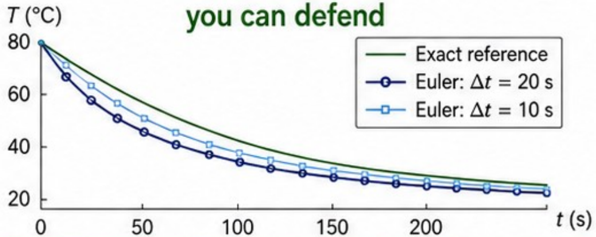
Euler method
- $\Delta t = 20$ s
- $\Delta t = 10$ s

Exact solution (reference)

Check a limitation you can justify
(step size, domain, assumptions)

3. Evidence

Create and check evidence you can defend



Method	$T(200\text{ s})$ (°C)	Absolute error (°C)
Exact (reference)	28.1201	—
Euler, $\Delta t = 20$ s	26.4425	1.6777
Euler, $\Delta t = 10$ s	27.2946	0.8255

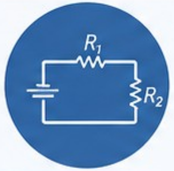
Document the required check, your chosen check, limitation and rationale

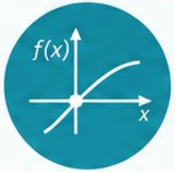
One question. Known methods. | Careful computation. | Defensible evidence.

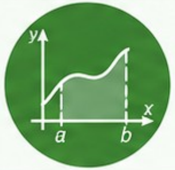
Name the Output Before Choosing the Method

Start with the question. Let the output you want guide the method you use.

What are you trying to find?	Method that matches (familiar tools)
What are the currents in each branch of this circuit? (several unknowns at once)	**Linear system** Set up linear equations and solve. (e.g., MATLAB backslash \ )
At what position does the ball's height first reach zero? (find the root of a function)	**Root finding** Solve $f(x) = 0$ for x. (e.g., MATLAB `fzero`)
What is the total work done from *a* to *b*? (accumulated amount over an interval)	**Integration** Accumulate area under a curve. (e.g., MATLAB `integral` or `trapz`)
How does the temperature change over time? (state evolves with time)	**ODE stepping** Step an initial-value ODE forward in time. (e.g., Euler, ode45, custom stepper)

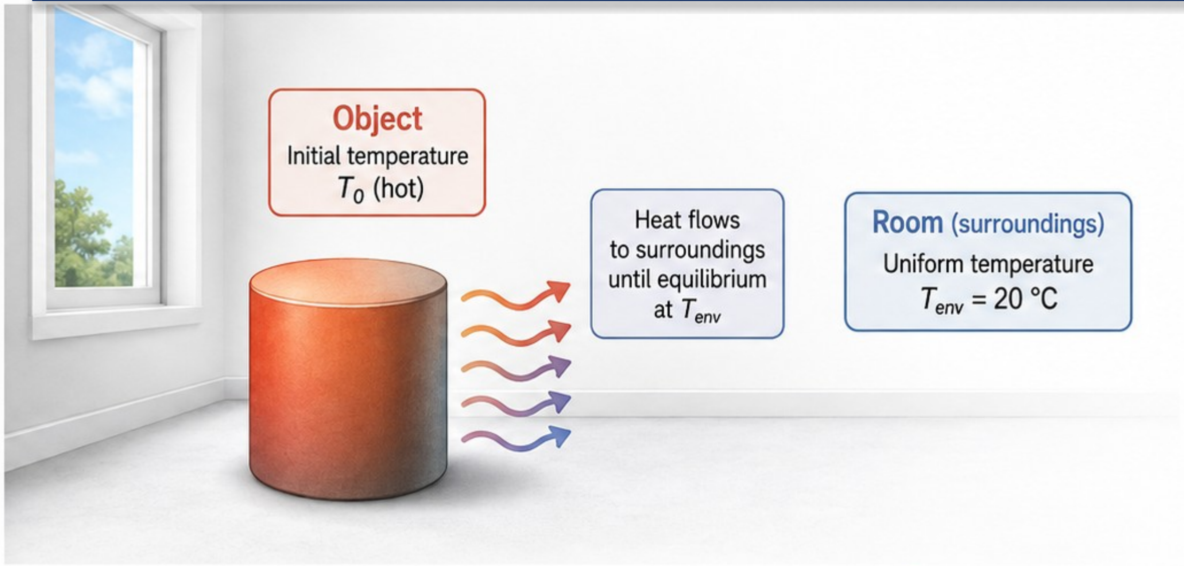






Name the output first → match the method → check the limitation.

Inputs, Output, and One Limitation



Inputs:
T_0, T_{env}, *tau* and times

Output:
temperature versus time

Assumptions:
uniform object and constant surroundings

Limitation:
finite timestep and model assumptions

Integrated Method Recognition
Identify and connect the method inside the model

→

Supplied-Code Auditing
Read, trace, and verify the implementation

→

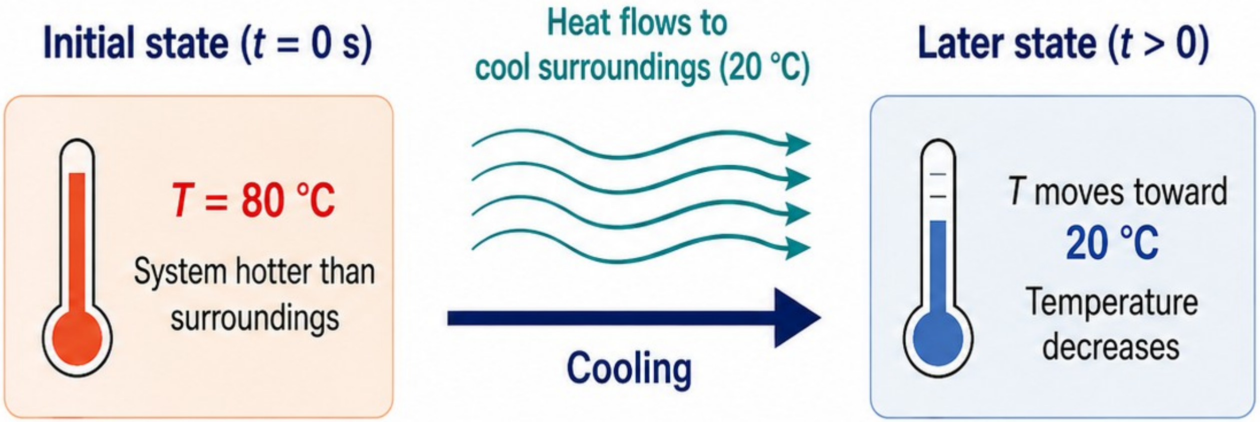
Validation
Check results against the locked reference to confirm the implementation

→

Formative Capstone Evidence Assembly
Document, reflect, and assemble your evidence

Predict the Cooling Before Running MATLAB

Use the model first. Do not wait for the plot.



Prediction: Temperature must decrease from 80 °C toward 20 °C over time.

Prediction Checklist

- Start hotter than the surroundings
- Decrease toward 20 °C
- Initial value is 80 °C
- A rising curve contradicts this model

If your output shows a rise, pause. Re-check the model, sign, and first update.

Connect the Rate to the Update

Newton Cooling Law

$$\frac{dT}{dt} = -\frac{(T - T_{\mathrm{env}})}{\tau} \rightarrow$$

Rate: $-(T-T_{env})/tau$

Rate units: deg C per second

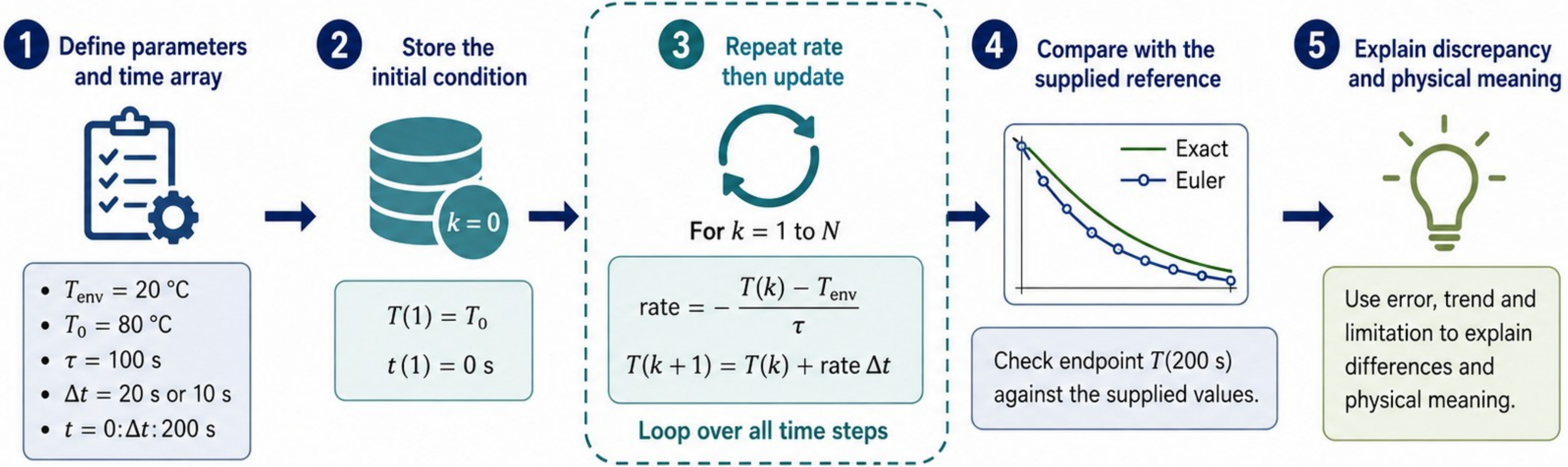
Euler Update

$$T_{next} = T_{current} + dt * \Bigg[\overbrace{-\frac{(T_{current} - T_{env})}{\tau}}^{\text{deg C / s}}\Bigg] \rightarrow \text{deg C / s} * \underbrace{\text{s}}_{dt \text{ units}} = \underbrace{\text{deg C}}_{\text{temperature change}}$$

1. Multiply by dt to get a temperature change
2. Add the change to the current state

Read the Algorithm Before the Code

Make the computational recipe explicit before you write or audit MATLAB.



1. **Define parameters and time array** — Set T_{env}, T_0, τ, Δt and the time vector t.
2. **Store the initial condition** — Record the known starting temperature at $t = 0$.
3. **Repeat rate then update** — Compute the rate, update T, and advance the loop over all steps.
4. **Compare with the supplied reference** — Validate against the exact or higher-fidelity reference.
5. **Explain discrepancy and physical meaning** — Interpret error, trend and limitation in context.

Trace One Euler Step

MATLAB code:

```
T_C(1)=T0;
dTdt=-(T_C(1)-Tenv)/tau;
T_C(2)=T_C(1)+dt*dTdt;
```

Given values:

T0 = 80, Tenv = 20, tau = 100 s, dt = 20 s

Trace of T_C through one Euler step

index	time	temperature
1	0 s	80 deg C
2	20 s	68 deg C

Index 1 represents $t = 0$ (initial condition).

Index 2 is the first updated value at 20 s.

One Euler step calculation:

Initial rate –0.6 deg C/s → Change –12 deg C → New temperature 68 deg C

Key points / trace facts:

- T(1)=80 at time zero
- dt=20 s
- Initial rate=–0.6 deg C/s
- Change=–12 deg C
- T(2)=68 deg C at 20 s

T_C(1)=80 at t=0. After one Euler step with dt=20 s, the first updated value is T_C(2)=68 deg C at t=20 s.

A Script Can Run and Still Be Wrong

DEFECTIVE

```
% Newton cooling (DEFECTIVE SIGN)
Tenv = 20;
T = 80;
tau = 100;
dt = 20;

dT = +(T - Tenv)/tau;    % WRONG sign
Tnext = T + dt*dT;
```

1. **Plus-sign rate gives 92 deg C on the first step**

$$dT = +(80 - 20)/100 = +0.6 \text{ deg C/s}$$

$$T_{next} = 80 + 20(0.6) = \mathbf{92 \text{ deg C}}$$

2. **The result moves away from the room temperature**

Locked model facts

Newton cooling $\frac{dT}{dt} = -\frac{(T - T_{env})}{\tau}$

$T_{env} = 20$ deg C

$T_0 = 80$ deg C

$\tau = 100$ s

Euler $dt = 20$ s

1. Repair the sign in the rate
2. Rerun and validate independently

REPAIR

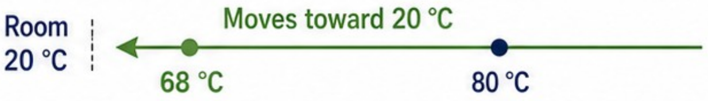
```
% Newton cooling (CORRECT SIGN)
Tenv = 20;
T = 80;
tau = 100;
dt = 20;

dT = -(T - Tenv)/tau;    % CORRECT sign
Tnext = T + dt*dT;
```

1. **Minus-sign rate gives 68 deg C on the first step**

$$dT = -(80 - 20)/100 = -0.6 \text{ deg C/s}$$

$$T_{next} = 80 + 20(-0.6) = \mathbf{68 \text{ deg C}}$$

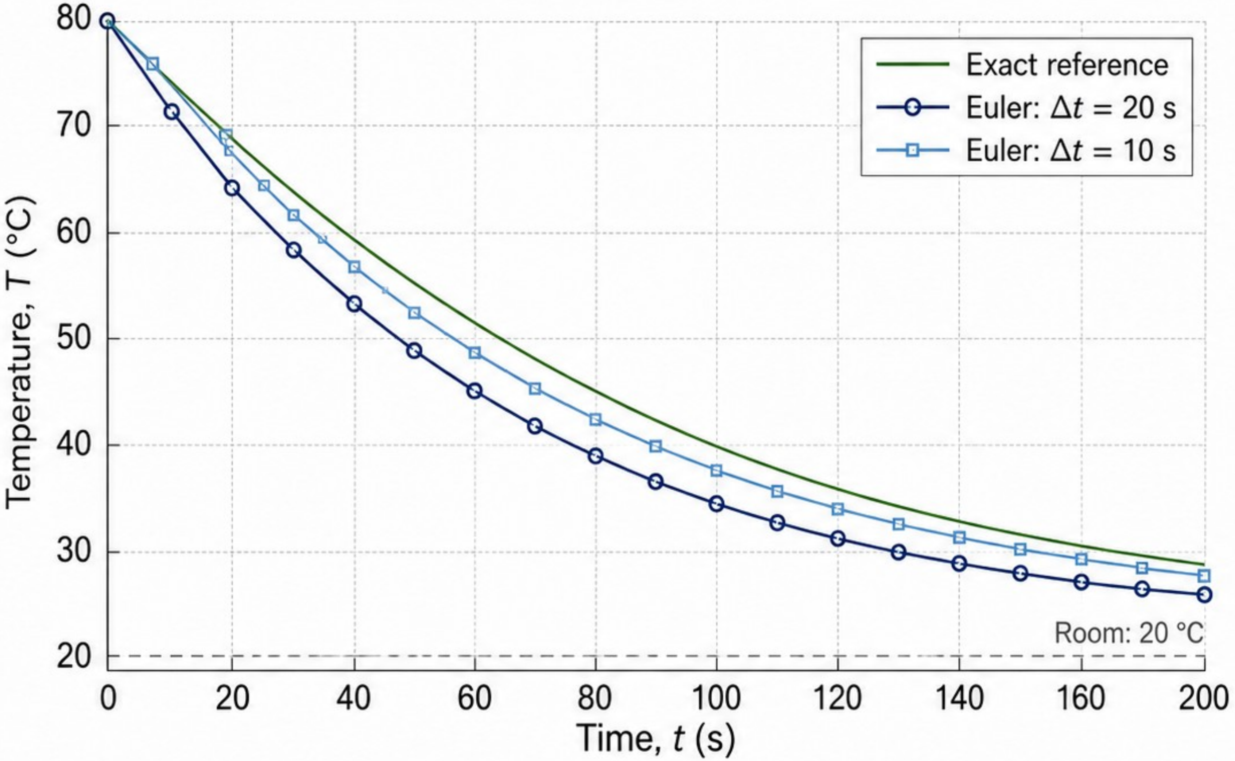
2. **The result moves toward the room temperature**



Syntactically valid code can still be physically wrong. Fix the model sign and validate independently.

Validate Against the Supplied Reference

Same model, same final time: 200 s



At t = 200 s

Exact reference

28.1201 °C

Euler step	Temperature	Absolute error
20 s	26.4425 °C	1.6777 °C
10 s	27.2946 °C	0.8255 °C

The finer step is closer to the supplied reference.

This checks the numerical result for the stated model—not every real-world assumption.

Choose a Check That Tests the Claim

Pick the validation check whose purpose matches the claim you want to test.

Initial condition tests setup

Confirms the model starts from the intended state and parameters.

Reference comparison tests numerical output

Compares the numerical result to a trusted reference at the same time.

Timestep refinement tests discretisation sensitivity

Repeats the calculation with smaller steps to see if the result changes systematically.

Units alone cannot catch every sign defect

A result can have correct units yet be wrong due to a sign error.

Example: aligning a claim with the right check

Claim: Euler cooling is close enough → **Check:** compare with supplied exact endpoint

A clear **claim** leads to the right **check**—and a meaningful result.

Change One Parameter and Explain the Physics

What we will do

- Change tau from 100 s to 150 s
- Hold other inputs fixed
- Predict slower cooling
- Compare with the corresponding exact reference

Parameter	Symbol (units)	Value (held fixed)
Initial temperature	T_0 (°C)	80 °C
Ambient temperature	T_∞ (°C)	20 °C
Time step	Δt (s)	20 s
Total time	t_{final} (s)	200 s
Time constant (changed)	τ (s)	100 s → 150 s

All inputs other than tau are fixed.

Controlled change

BEFORE

tau = 100 s

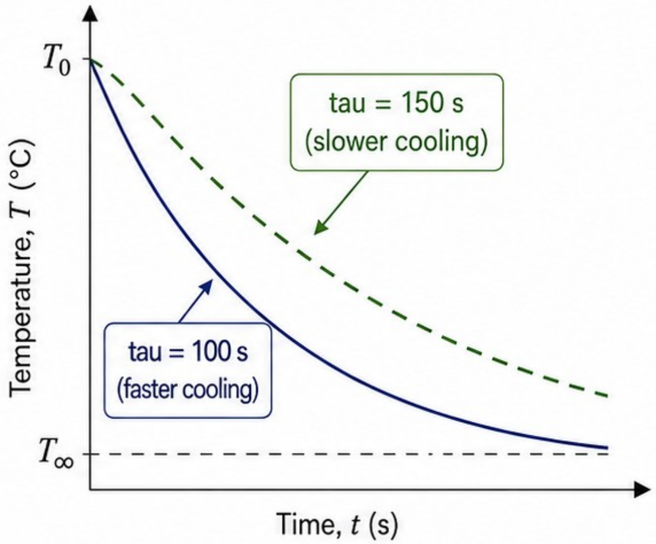
All other inputs fixed

AFTER

tau = 150 s

All other inputs fixed

Conceptual cooling response



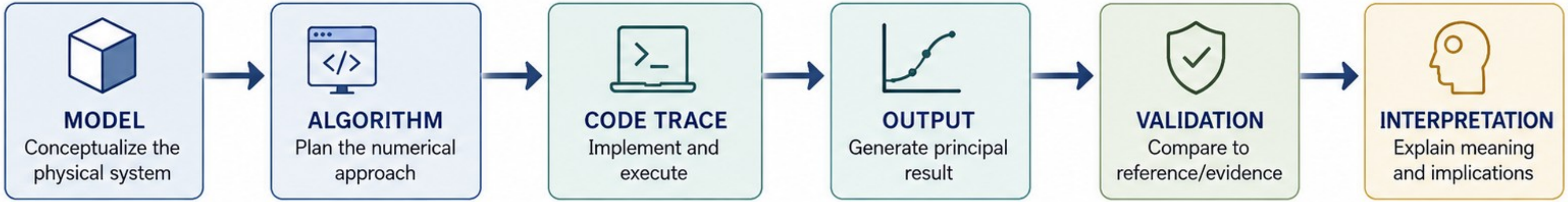
Larger tau means a slower approach to ambient temperature.

The lecturer MATLAB script computes and checks this modification.

It generates the numerical result for tau = 150 s and compares it with the corresponding exact reference.

PHY4605 Week 12: Assemble the Capstone Evidence

Build and document the complete physical-model-to-interpretation chain.



1	**State model, assumptions, variables and units**	REQUIRED	Clearly describe the physical model and assumptions. List all variables with their symbols and units.
2	**Show pseudocode and one justified modification**	REQUIRED	Provide pseudocode for your algorithm. Name one modification and justify why you made it.
3	**Include one principal output**	REQUIRED	Show your primary result with proper label, units, and a concise caption.
4	**Include required plus chosen validation**	REQUIRED CHOSEN	Perform the required validation. Add one additional, chosen validation and report it.
5	**Explain one limitation**	REQUIRED	Identify one important limitation of your model, method, or data, and explain its impact.
6	**Provide complete evidence package**	CHOSEN	Submit all source files, data, and outputs used to produce your results.
7	**Document key decisions**	CHOSEN	Summarize key modeling and coding decisions and how they affect the results.
8	**State next steps**	CHOSEN	Outline one improvement or extension you would pursue next and why.

Reproduce and Rehearse the Defence

Fresh-Session Reproducibility Checklist

- Rerun from a fresh MATLAB session
- Record release, parameters and numerical settings
- Declare accepted, modified and rejected AI assistance
- Each member traces code and explains the physics

Two/Three-Person Individual Defence Rehearsal Path

1. Individual Run-Through

Each member presents their part end-to-end.

2. Cross-Check & Challenge

Peers ask probing questions, challenge assumptions and verify results.

3. Full Defence Simulation

Take turns as presenter, questioners and moderator. Refine until answers are crisp, evidence-backed and consistent.

What Makes Your Result Trustworthy?

METHOD

1. Name one method from its required output

CHECK

2. Identify one code step and validation result

NEXT ACTION

3. State the capstone item still needing attention